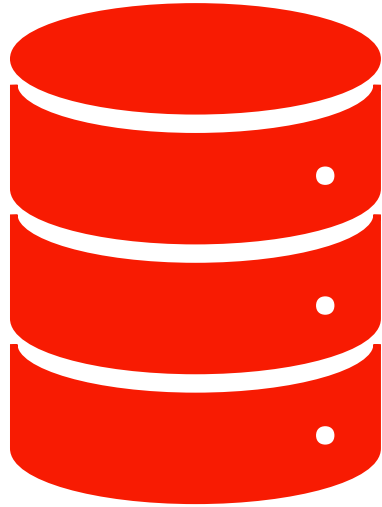


ALGORITMOS DE CLASIFICACIÓN



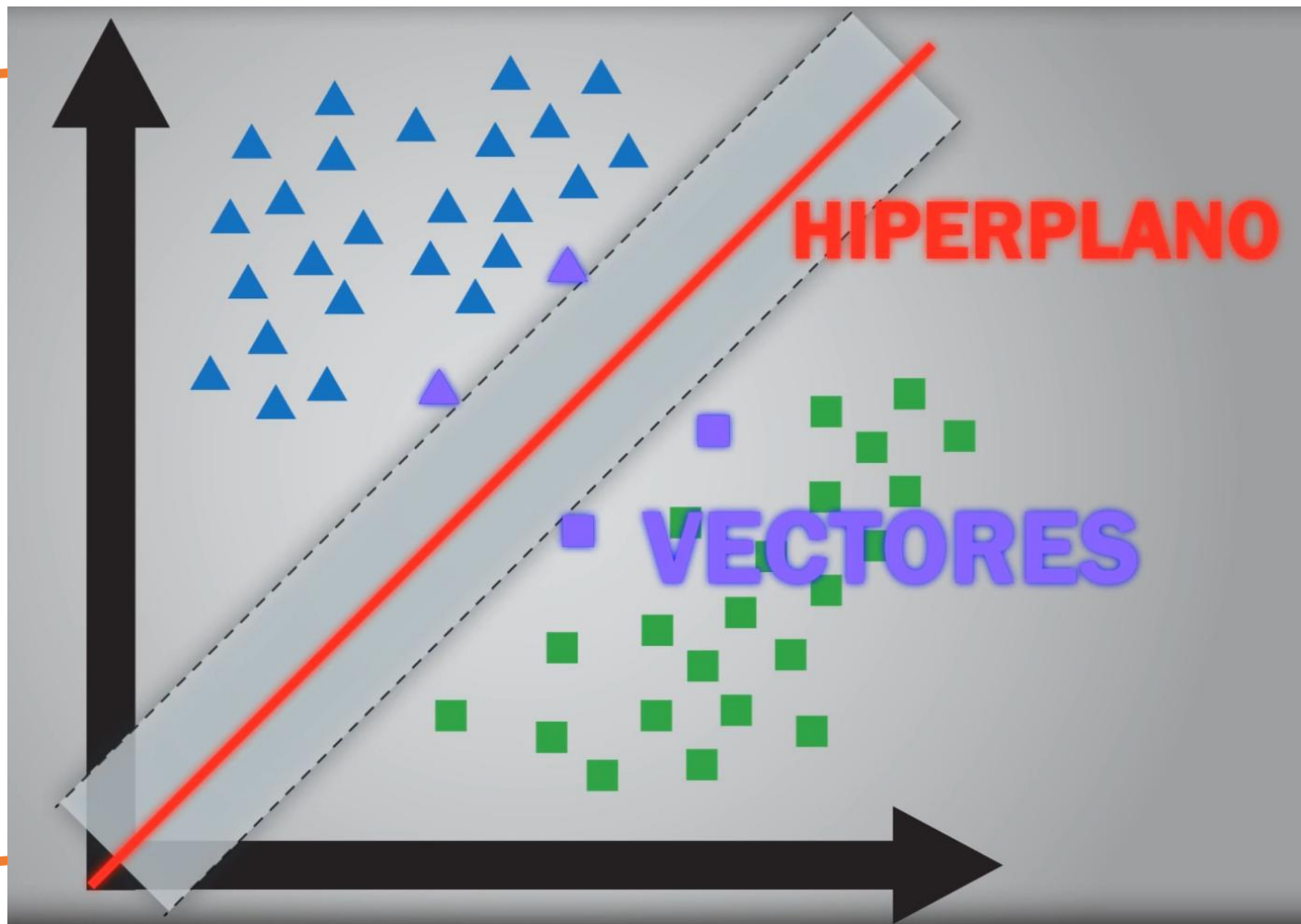
Máquina de Vetores de Soporte (SVM)

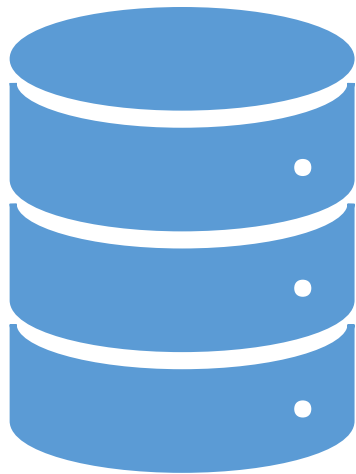
¿Qué es una SVM?

Una **Máquina de Vectores de Soporte (SVM)** es un modelo de aprendizaje supervisado que se usa para **clasificación** y, en menor medida, para **regresión**.



La idea principal es encontrar el **hiperplano** que **mejor separa las clases** en un espacio de características.





¿Qué hace una SVM?

- SVM es un modelo que **aprende a clasificar** dividiendo el espacio de datos en regiones: una para cada clase.
- Por ejemplo: supongamos que queremos clasificar si un animal es un **gato** o un **perro** en base a su peso y tamaño. Cada punto en el plano (peso, tamaño) representa un animal. SVM busca **la mejor línea (o hiperplano) que separa claramente a los gatos de los perros**.
- 🧭 ¿Por qué "la mejor"?
- Porque entre todas las posibles líneas, elige **la que deja más espacio libre entre sí misma y los puntos más cercanos de cada clase**. Es decir, **maximiza el margen**.

<https://www.youtube.com/watch?v=kl6tyEi5eso>

Componentes importantes

1. Hiperplano

2. Margen

3. Vectores de soporte

4. Parámetro C

5. Kernel

¿CUÁNDO USAR SVM?

Usar SVM si:

- Tienes datos **de pocas dimensiones** (funciona bien en 2D, 3D o incluso con 10-100 features)
- No tienes **demasiados datos** (más lento que otros modelos en big data)
- Quieres **precisión** y tienes datos **limpios**
- Las clases están **bien separadas** o sabes que un kernel puede ayudarte a separarlas

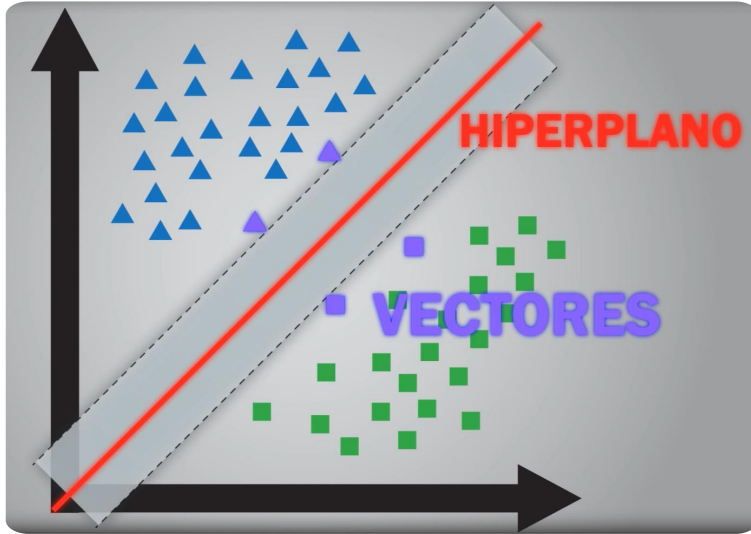
Evitar SVM si:

- Tienes millones de datos → usa Random Forest, XGBoost o redes neuronales
- Necesitas interpretabilidad clara → SVM no es tan interpretable como regresión lineal

**SVM no te
dice
directamente
qué variable
influye y cómo.**

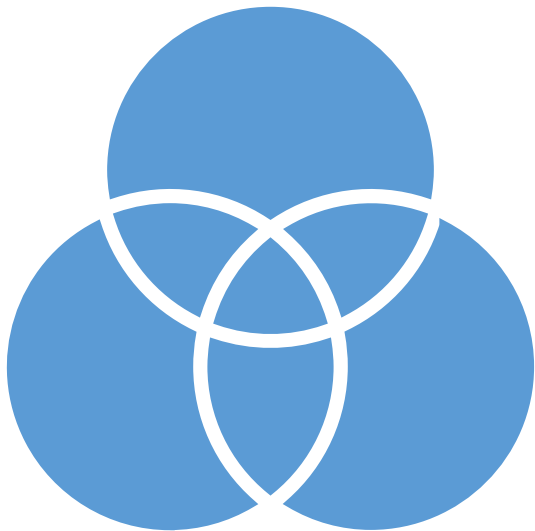
- El modelo toma decisiones **basado en distancias y márgenes**, no en pesos lineales fácilmente legibles.
- Aún en el caso de SVM lineal, los coeficientes no tienen una interpretación intuitiva como en regresión lineal.
- Y si usas **kernels no lineales (como RBF)**, es todavía **menos interpretable**, porque estás transformando los datos a un espacio que **ni tú puedes ver**.

Intuición



- Tenemos datos de dos clases (por ejemplo, puntos verdes y azules) distribuidos en un plano 2D. Hay muchas líneas que podrían separar esas dos clases, pero SVM busca **la que tenga el mayor margen**, es decir, la **distancia máxima entre la línea y los puntos más cercanos de cada clase**.
- Esos puntos más cercanos se llaman **vectores de soporte**.

¿Cómo lo hace?



- **Encuentra un hiperplano** que separa las clases.
- **Maximiza el margen** entre las clases.
- Si los datos **no son linealmente separables**, SVM puede:
 - Usar un **margen suave (soft margin)** y permitir errores.
 - Usar un **truco del kernel** para mapear los datos a un espacio de mayor dimensión donde sí son separables.

Componentes importantes

1. Hiperplano

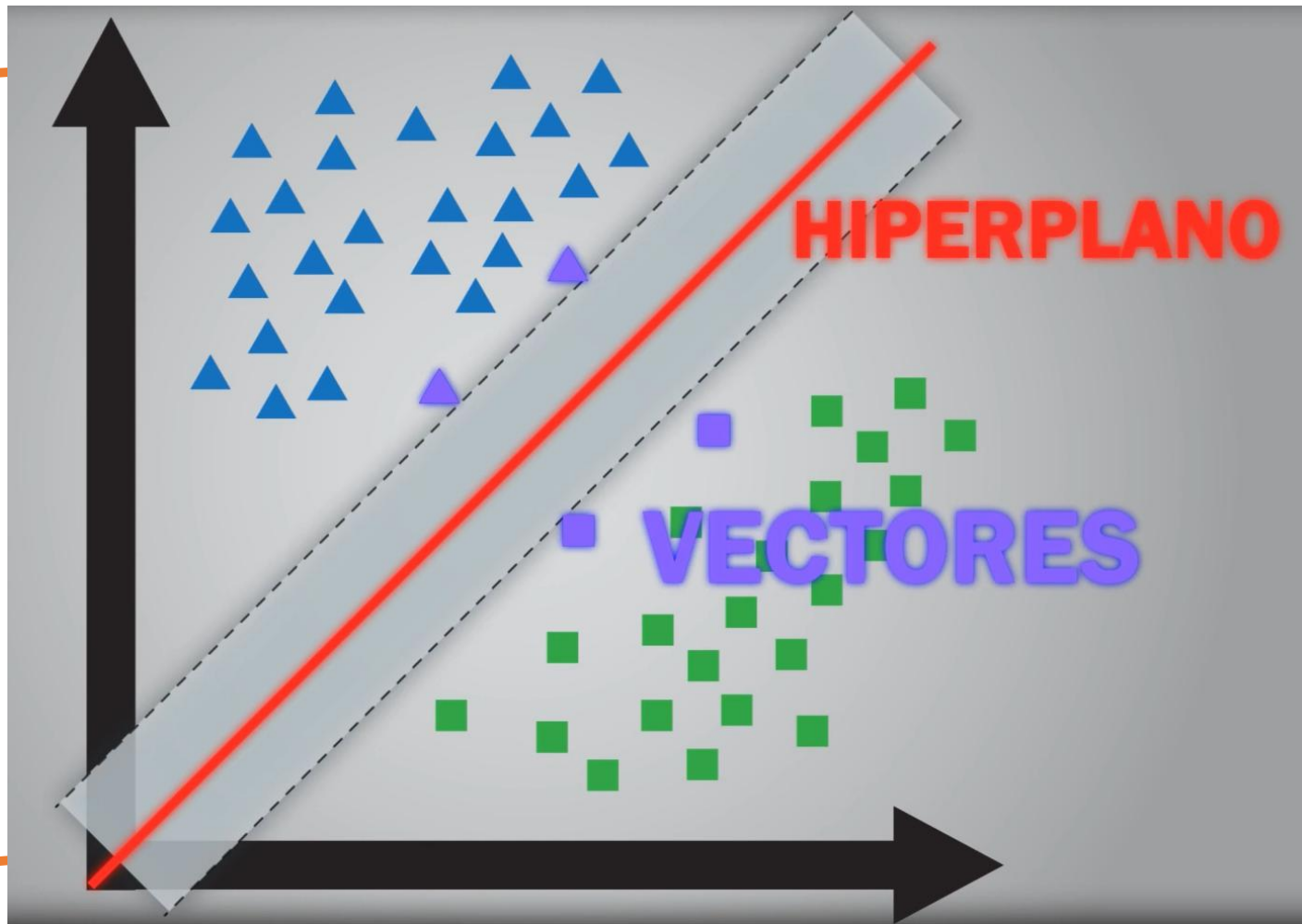
- En 2D es una línea, en 3D es un plano, en dimensiones mayores es un "hiperplano".

2. Margen

- La distancia entre el hiperplano y los vectores de soporte más cercanos.

3. Vectores de soporte

- Los datos que “definen” el margen. Son los más críticos del conjunto.



Componentes importantes

4. Parámetro C

- Controla el **trade-off entre maximizar el margen y minimizar errores de clasificación**.
- Si C es grande → penaliza más los errores (modelo más rígido).
- Si C es pequeño → permite más errores para lograr mayor margen (modelo más flexible).

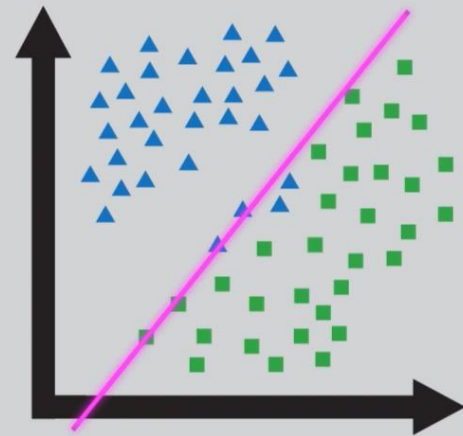
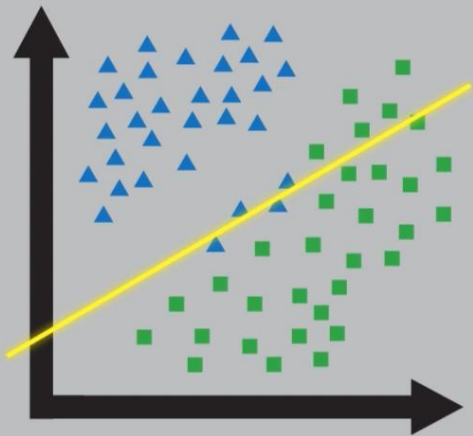
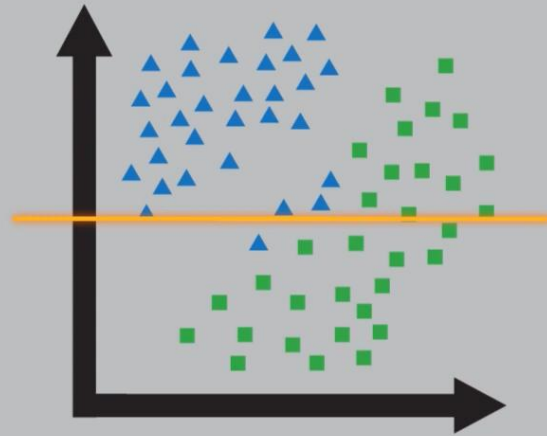
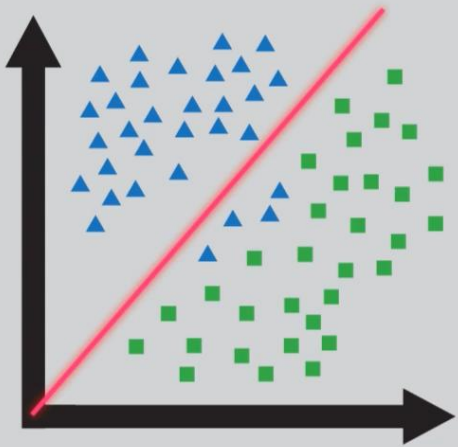
5. Kernel

- Permite a SVM trabajar con **datos no lineales** al transformarlos en un espacio donde sí sean linealmente separables.
- Ejemplos: **Lineal, Polinómico, RBF (Radial Basis Function / Gaussiano)**



¿Qué es el parámetro C en SVM?

- 
- C es como el **nivel de exigencia** del modelo. Es decir, **cuánto le permites equivocarse** durante el entrenamiento.



¿Qué es el parámetro C en SVM?

En clasificación con SVM (modelo SVC), C es un **hiperparámetro de regularización** que controla el compromiso entre:

Maximizar el margen (hacer el modelo más general),

y **minimizar el error de clasificación en los datos de entrenamiento.**

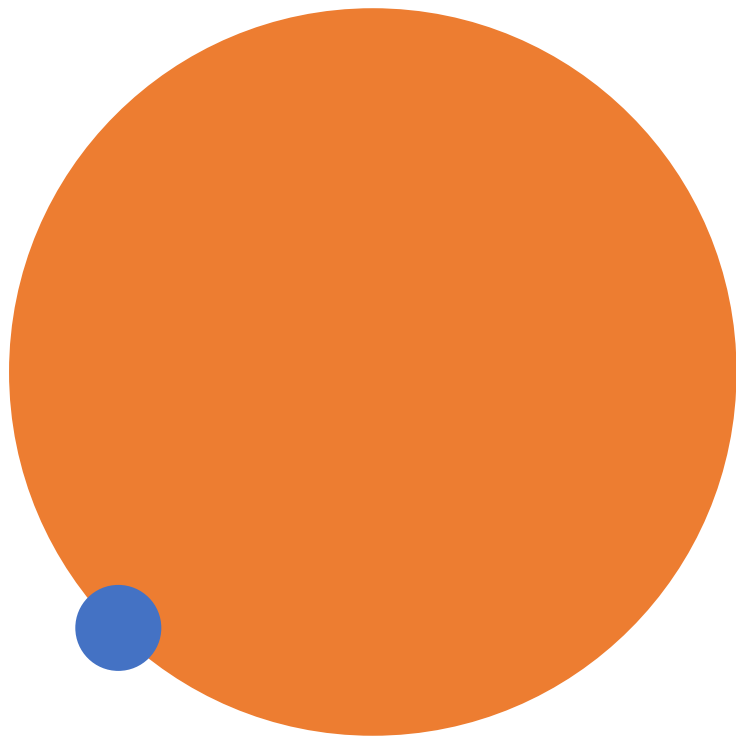
Pensemos en **C**
como un profe
corrigiendo
exámenes:

**C grande ($C = 1000$) → profe estricto,
no acepta errores**

- Quiere que **todos los puntos queden bien clasificados**.
- Puede terminar **memorizing** (sobreajuste), como un estudiante que repite sin entender.

**C pequeño ($C = 0.01$) → profe
relajado, **permite errores pequeños****

- Prefiere un modelo que **generalice bien**, aunque se equivoque un poco en entrenamiento.
- Resultado: **modelo más flexible y robusto**.



- Con $C = 1000$, SVM **intentará forzar** una línea que pase justo para clasificar bien todo, incluso si eso implica una frontera loca.
- Con $C = 0.1$, SVM dirá: “ok, dejen pasar este error, pero trazo una línea más estable para el resto”.



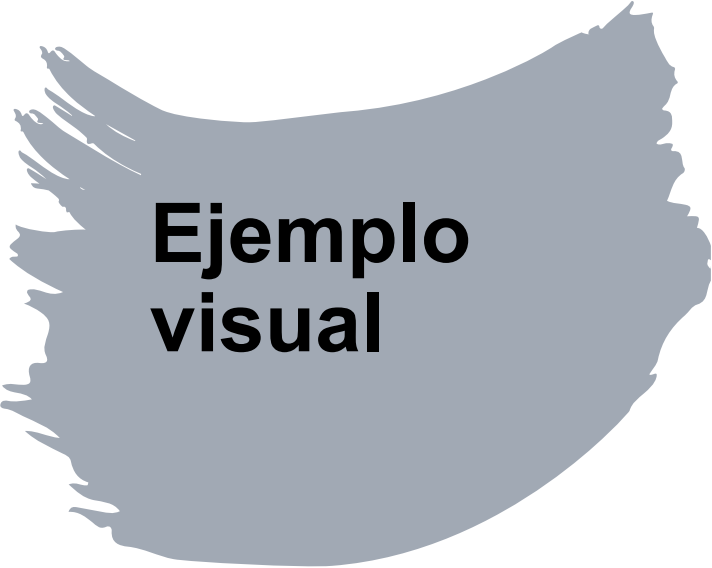
-
- ¿Qué pasa si no eliges bien C?

Valor de C	Qué pasa
Muy alto	Modelo sobreajusta (memoriza el ruido)
Muy bajo	Modelo subajusta (no capta bien el patrón)
Intermedio	Buen equilibrio entre margen ancho y pocos errores

Parámetro C

- Controla el equilibrio entre:
 - **Maximizar el margen**
 - **Clasificar bien los datos (evitar errores)**

Valor de C	Efecto
Alto (C = 1000)	Prefiere clasificar bien aunque el margen sea pequeño → puede sobreajustar
Bajo (C = 0.1)	Permite errores si puede aumentar el margen → modelo más general



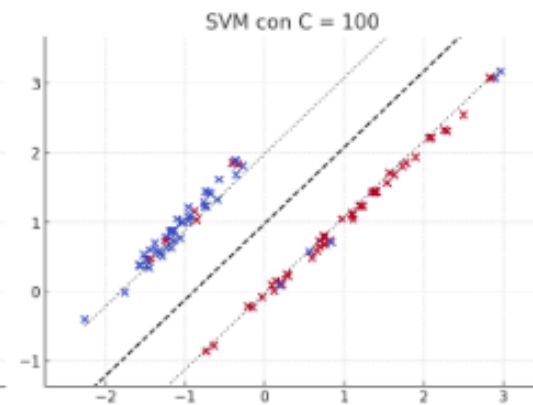
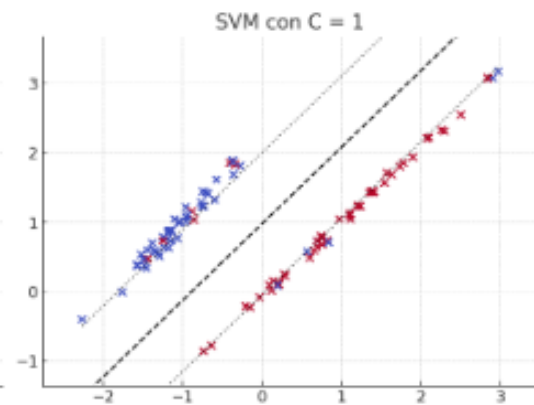
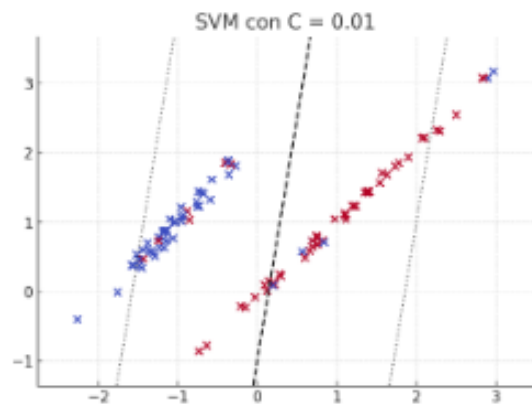
Ejemplo visual

Tenemos dos grupos de puntos con algo de ruido.

Si $C = 100$, el modelo buscará una línea que **pase justo por donde separa perfectamente**, aunque tenga que hacer una curva rara o reducir el margen.

Si $C = 0.01$, dirá: “no importa que algunos puntos estén mal clasificados, voy a enfocarme en separar **en general** con una buena distancia entre clases”.

Efecto del parámetro C en SVM (con kernel lineal)



Panel 1: $C = 0.01$	Panel 2: $C = 1$	Panel 3: $C = 100$
(modelo relajado, flexible) El modelo tolera errores: permite que algunos puntos estén mal clasificados. Busca márgenes más amplios entre las clases. Puede ser bueno para generalizar, pero no capta bien outliers o detalles finos.	(balance) Es un compromiso entre rigurosidad y flexibilidad. Aún permite algunos errores, pero no tantos como con $C = 0.01$. Buen punto de partida para muchos datasets reales.	(modelo exigente) El modelo intenta clasificar todo perfectamente, incluso si eso reduce el margen. Se ajusta más a los datos de entrenamiento. Riesgo: puede sobreajustarse y no generalizar bien en datos nuevos.

¿Cómo afecta eso al caso?

Con **C alto**, el modelo de SVM puede querer **predecir bien todos los datos de entrenamiento**, pero puede **fallar más en test** si hay ruido → **overfitting**.

Con **C bajo**, el modelo será más **robusto y general**, pero puede **cometer más errores en entrenamiento** → **underfitting** si te pasas.

Intuición: “la tolerancia al error”

Valor de C	Comportamiento del modelo
Alto (C grande)	! <i>Poca tolerancia al error</i>: el modelo intenta clasificar todo bien en entrenamiento, incluso si eso implica un margen más estrecho. Puede sobreajustarse (overfitting).
Bajo (C pequeño)	Más tolerancia al error: el modelo permite que algunos puntos estén mal clasificados si eso permite un margen más amplio. Esto ayuda a generalizar mejor (menos overfitting).

Kernel Trick

A veces los datos no se pueden separar con una línea recta. En este caso, se puede usar una técnica llamada “**kernel trick**” para transformar el espacio de datos a otro donde sí puedan separarse con una línea.

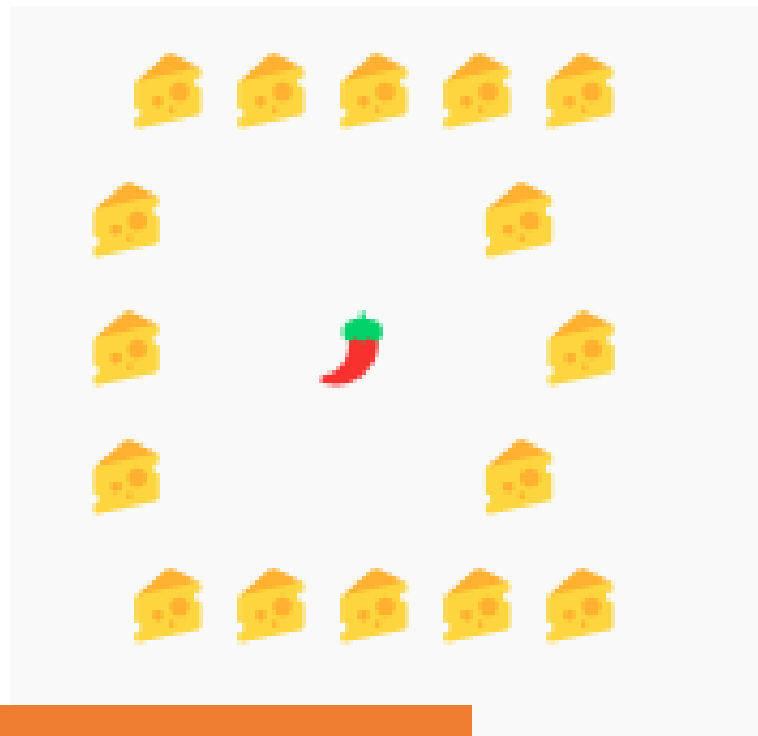
- Tipos comunes de kernel:

Kernel	Cuándo usarlo
Lineal	Cuando los datos se ven separables con una línea
Polinómico	Cuando hay relaciones más complejas
RBF (Radial / Gaussiano)	Cuando no se conoce bien la forma de la separación

¿Qué es el *Kernel Trick*?

Imaginemos lo siguiente:

- Tienes una tortilla 🍳 (plano 2D) y encima pones dos tipos de ingredientes:
- Guiso 🌶️ en forma de círculo
- Queso 🧀 en forma de anillo alrededor del guiso



Problema:

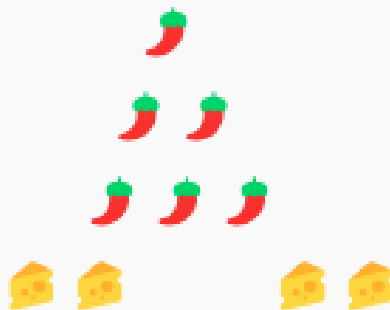
¡No hay forma de dibujar una línea recta que separe el queso del guiso en esa tortilla!

No son separables linealmente.

Solución: Kernel Trick = "levantar" la tortilla

- Ahora, desde arriba no podías separarlos...¡Pero en este espacio elevado sí puedes!

Vista en 3D:



Una SVM puede dibujar una "hoja" (hiperplano) en 3D y separar queso de guiso perfectamente.

¿Cómo se aplica esto en SVM?

- SVM usa **funciones kernel** para hacer eso automáticamente:
- **Toma tus datos originales y los transforma internamente a un espacio más grande y separable.**
- Tú no tienes que hacer nada, el kernel se encarga de esa "magia".

Tipos de Kernel

El kernel trick es como cambiar de perspectiva: si no puedes separar puntos en 2D, imagina que los subes a 3D donde sí puedes. Y lo mejor: SVM hace ese cambio sin que tú tengas que hacerlo manualmente.

Kernel	¿Para qué sirve?
Lineal	Cuando los datos sí se pueden separar con una línea recta
Polinómico	Para datos con curvas más suaves
RBF (Gaussiano)	Cuando no tienes idea de cómo se separan los datos, es el más flexible
Sigmoide	Inspirado en redes neuronales (menos usado)

¿Cómo decidir cuál usar?

1. Si tienes muchas features (más de 20):

- Usa "linear" → no necesitas transformar el espacio mucho, ya hay bastante dimensión.

2. Si tienes pocos datos o estás empezando:

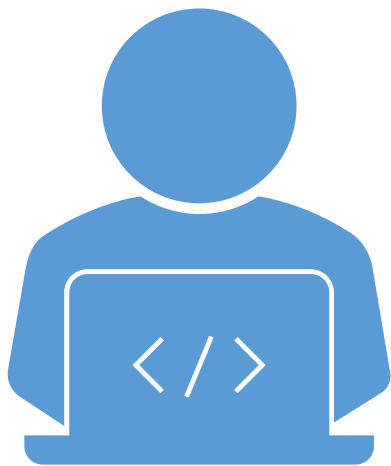
- Usa "rbf" → es un **kernel "por defecto" muy robusto** para muchos casos reales.

3. Si notas que "rbf" tarda mucho o sobreajusta:

- Prueba con "linear" o con "poly" de bajo grado (ej. degree=2 o 3).

4. Si tus datos tienen una forma circular, espiral o rara:

- Usa "rbf" → fue hecho para ese tipo de situaciones.



Consejo para clase

Empiecen con "rbf".
Si el modelo tarda mucho o
sobreajusta, prueben "linear".
Si saben que hay una relación
polinómica (por ejemplo,
ingresos crecen al cuadrado),
prueben "poly".

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# -----
# 1. Cargar datos
# -----
df = pd.read_csv("ProductPriceIndex.csv")

# -----
# 2. Limpiar columnas de precios
# -----
price_cols = ['farmprice', 'atlantaretail', 'chicagoretail', 'losangelesretail', 'newyorkretail']
for col in price_cols:
    df[col] = df[col].replace('[\$,]', '', regex=True).replace('', np.nan).astype(float)

df.dropna(subset=price_cols, inplace=True)

# -----
# 3. Crear etiqueta binaria
#     (1 = precio en NY alto, 0 = bajo)
# -----
ny_median = df['newyorkretail'].median()
df['ny_high'] = (df['newyorkretail'] > ny_median).astype(int)

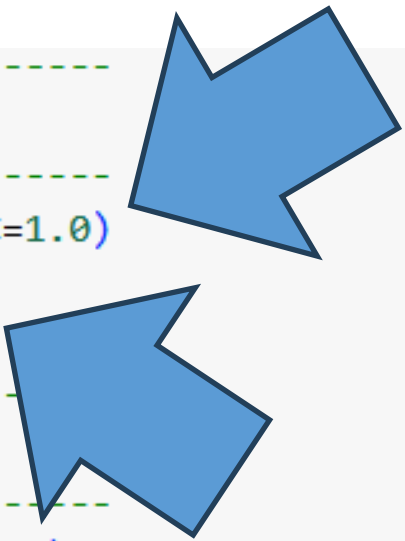
# -----
# 4. Preparar features y target
# -----
features = ['farmprice', 'atlantaretail', 'chicagoretail', 'losangelesretail']
X = df[features]
```

```
# 5. Train/test split
# -----
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

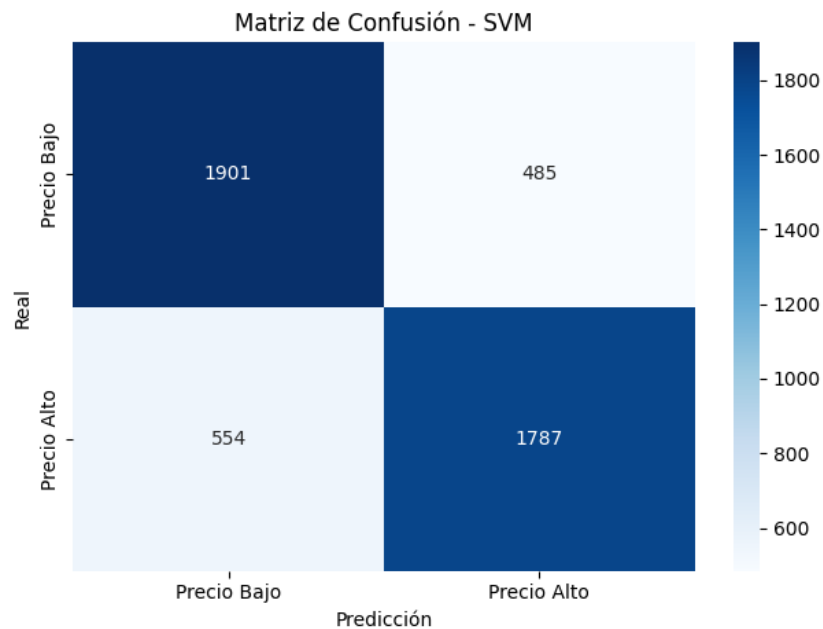
# -----
# 6. Entrenar modelo SVM
# -----
model = SVC(kernel='rbf', C=1.0)
model.fit(X_train, y_train)

# -----
# 7. Evaluar modelo
# -----
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2%}")
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred))
```

```
# -----  
# 6. Entrenar modelo SVM  
# -----  
model = SVC(kernel='rbf', C=1.0)  
model.fit(X_train, y_train)  
  
# -----  
# 7. Evaluar modelo  
# -----  
y_pred = model.predict(X_test)  
accuracy = accuracy_score(y_test, y_pred)  
print(f"Accuracy: {accuracy:.2%}")  
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred))
```

Two blue arrows with black outlines are positioned over the code. One arrow points from the top right towards the training code block (lines 4-6), and the other points from the bottom right towards the evaluation code block (lines 10-14).

```
# -----
# 8. Matriz de confusión (gráfico)
# -----
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Precio Bajo', 'Precio Alto'],
            yticklabels=['Precio Bajo', 'Precio Alto'])
plt.title("Matriz de Confusión - SVM")
plt.xlabel("Predicción")
plt.ylabel("Real")
plt.tight_layout()
plt.show()
```



MATRIZ DE CONFUSIÓN INTERPRETADA

	Predicción: Precio Bajo	Predicción: Precio Alto
Real: Precio Bajo	1901 (✓ aciertos)	485 (✗ falsos positivos)
Real: Precio Alto	554 (✗ falsos negativos)	1787 (✓ aciertos)

Verdaderos Negativos (1901)

Predijo “Precio Bajo” y realmente era Bajo

✓ Acertó con productos baratos.

Falsos Positivos (485)

Predijo “Precio Alto” pero realmente era Bajo

Error tipo 1: falsa alarma.

➡ El modelo pensó que ese producto sería caro... y no lo fue.

Esto puede llevar a acciones innecesarias como limitar stock, subir precios, etc.

Falsos Negativos (554)

Predijo “Precio Bajo” pero **realmente era Alto**

Error tipo 2: se le escapó un caso importante.

➡ El modelo no detectó un precio alto real.

Esto puede llevar a **perder oportunidades** (no ajustar logística, no promocionar).

Verdaderos Positivos (1787)

Predijo “Precio Alto” y realmente era Alto

✓ Acertó con productos costosos.

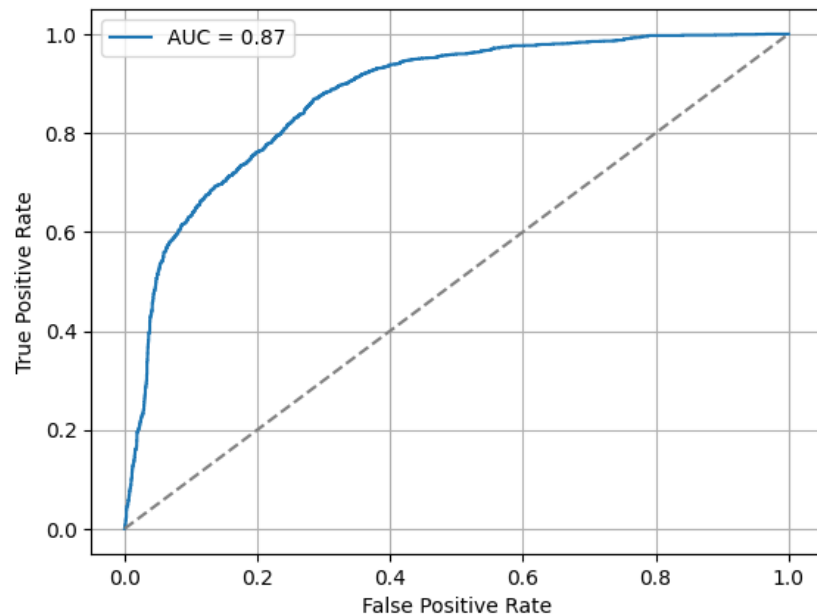
¿Qué significa esto en mi caso?

El modelo es bastante equilibrado: buena precisión y buen recall.

Comete más falsos negativos que falsos positivos → Se le escapan más precios altos que lo contrario.

Si para mi caso es crítico **no perder ningún precio alto** (ej: logística, promoción), podría **ajustar el umbral** para subir el recall, aunque se pierda algo de precisión.

Curva ROC - SVM



```
from sklearn.metrics import roc_curve, roc_auc_score

# Necesitas probabilidad de pertenecer a clase 1
y_scores = model.decision_function(X_test)

fpr, tpr, thresholds = roc_curve(y_test, y_scores)
auc = roc_auc_score(y_test, y_scores)

plt.plot(fpr, tpr, label=f"AUC = {auc:.2f}")
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Curva ROC - SVM")
plt.legend()
plt.grid()
plt.show()
```

¿Qué es la Curva ROC?

ROC = Receiver Operating Characteristic

Mide la capacidad del modelo para **distinguir entre clases** (en tu caso, “precio alto” vs “precio bajo”).

El eje X: **Tasa de Falsos Positivos (FPR)**

El eje Y: **Tasa de Verdaderos Positivos (TPR / Recall)**

Cuanto **más arriba y a la izquierda esté la curva**, mejor está separando las clases.

AUC: Área Bajo la Curva

$$\text{AUC} = 0.87$$


Eso significa que el modelo
tiene un **87% de
probabilidad** de clasificar
correctamente una muestra
aleatoria positiva frente a una
negativa.

Interpretación de resultado:

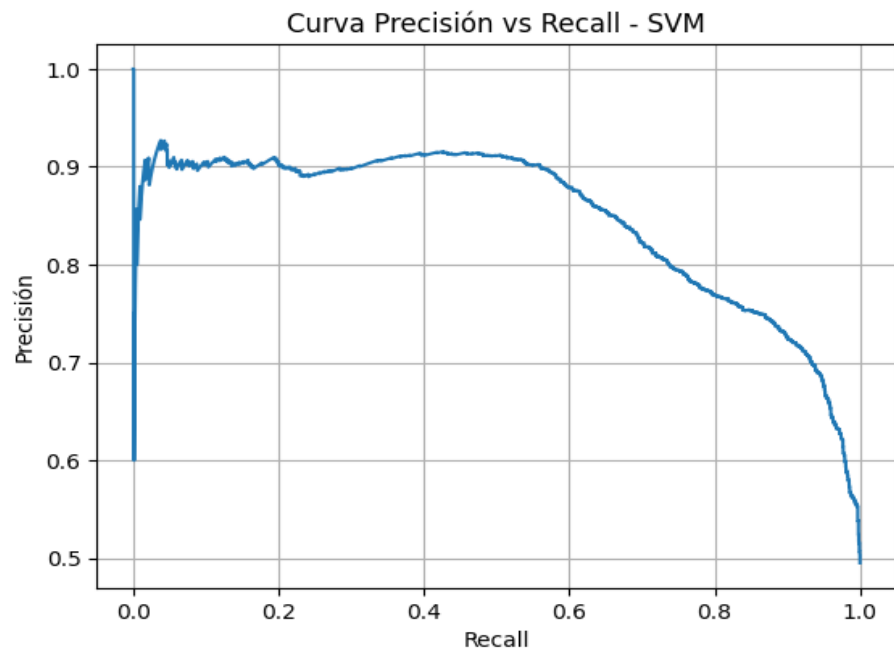
El modelo tiene un **AUC de 0.87**, lo que indica una **muy buena capacidad discriminativa**.

AUC Score	Interpretación
0.5	Modelo aleatorio (mala separación) ✖
0.6–0.7	Separación débil
0.7–0.8	Aceptable ✔
0.8–0.9	Buena separación
0.9–1.0	Excelente (ojo: cuidado con sobreajuste)

¿Qué hacer con este análisis?

- Refuerza que el modelo SVM **está funcionando bien** y no solo por "accuracy", sino por su **discriminación en umbrales**.
- Si se tuviera una clase más importante (ej. detectar precios altos para marketing o logística), se puede usar este gráfico para **ajustar el umbral de decisión**, no solo usar el 0.5 por defecto.





```
from sklearn.metrics import precision_recall_curve

precision, recall, thresholds = precision_recall_curve(y_test, y_scores)

plt.plot(recall, precision)
plt.xlabel("Recall")
plt.ylabel("Precisión")
plt.title("Curva Precisión vs Recall - SVM")
plt.grid()
plt.show()
```

¿Qué muestra la curva Precisión vs Recall?

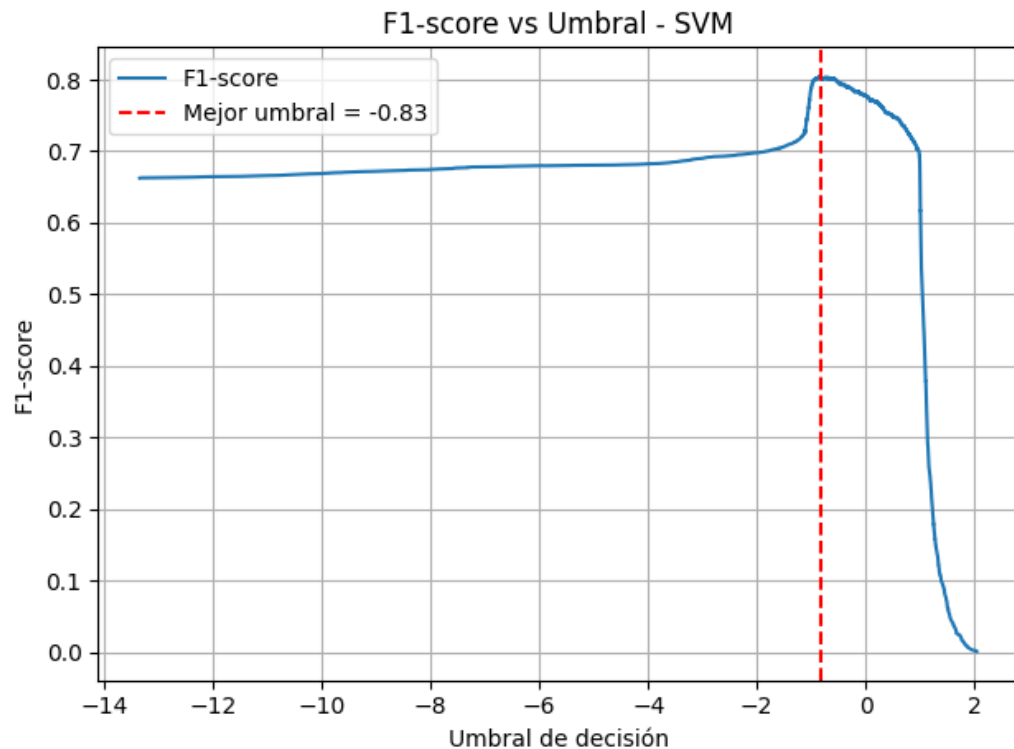
Eje X (Recall):
proporción de positivos
reales que el modelo
detecta correctamente
(también llamado
sensibilidad).

Eje Y (Precisión):
proporción de
predicciones positivas
que realmente eran
positivas.



¿Cómo leer la gráfica?

- **Valores altos de precisión y recall juntos** son ideales (zona superior derecha).
 - La gráfica muestra que:
 - Cuando el modelo **intenta capturar más positivos (recall → 1.0)**, lo hace sacrificando **precisión** (baja hasta ~ 0.5).
 - Cuando el modelo es más **cauteloso (recall bajo)**, logra **precisiones por encima de 90%**, lo cual es excelente.
-



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve, f1_score

# Obtener puntajes y etiquetas reales
y_scores = model.decision_function(X_test)

# Calcular precisión, recall y thresholds
precision, recall, thresholds = precision_recall_curve(y_test, y_scores)

# Calcular F1 para cada punto
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-6) # evitar división por cero

# Encontrar el mejor umbral
best_index = np.argmax(f1_scores)
best_threshold = thresholds[best_index]
best_f1 = f1_scores[best_index]

print(f"Mejor F1-score: {best_f1:.4f} en umbral: {best_threshold:.4f}")

# Graficar F1-score vs thresholds
plt.plot(thresholds, f1_scores[:-1], label="F1-score")
plt.axvline(best_threshold, color='r', linestyle='--', label=f"Mejor umbral = {best_threshold:.2f}")
plt.title("F1-score vs Umbral - SVM")
plt.xlabel("Umbral de decisión")
plt.ylabel("F1-score")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

¿Qué muestra este gráfico?

- **Eje X:** Umbral de decisión (decision_function).
Valores bajos → más predicciones positivas (más recall).
Valores altos → más exigente para clasificar como positivo (más precisión).
- **Eje Y:** F1-score para cada valor del umbral.
El F1-score es el **balance entre precisión y recall**.

Interpretación del gráfico

El máximo F1-score se alcanza en el umbral de ≈ -0.83 (línea roja).

Eso significa que si usamos **ese umbral** para decidir si algo es “precio alto”, se obtiene el **mejor equilibrio entre precisión y recall**.

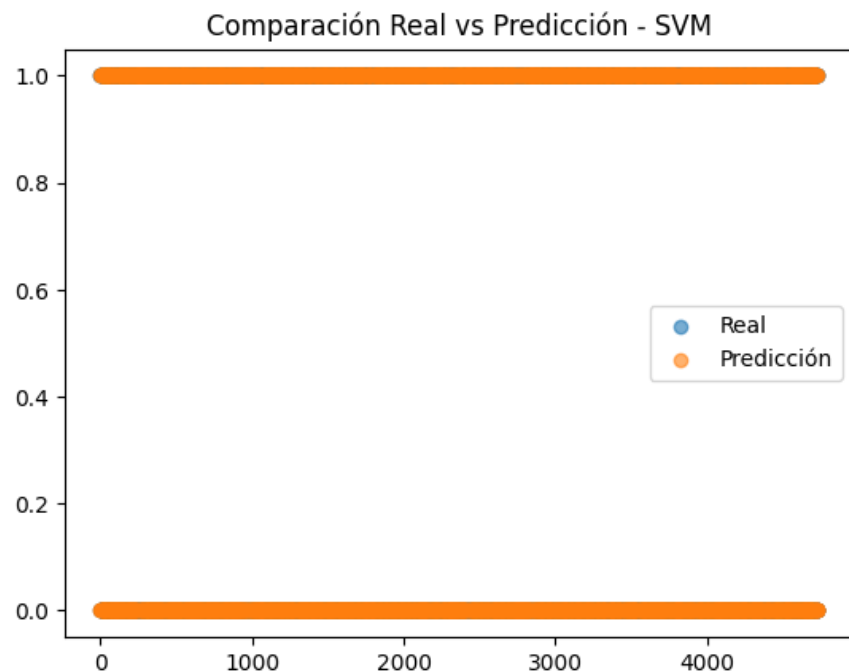
El F1-score en ese punto parece estar cerca de 0.8, lo cual es muy bueno.

**¿Por qué no
usar el
umbral por
defecto (= 0)?**

El modelo por defecto usa umbral = 0, pero el gráfico muestra que el rendimiento **mejora significativamente bajando ese umbral a -0.83.**

Eso indica que el modelo es un poco **conservador** por defecto y **necesita ser más "agresivo"** al clasificar como "precio alto".

```
plt.scatter(range(len(y_test)), y_test, label="Real", alpha=0.6)
plt.scatter(range(len(y_pred)), y_pred, label="Predicción", alpha=0.6)
plt.legend()
plt.title("Comparación Real vs Predicción - SVM")
plt.show()
```



¿Qué muestra este gráfico?

- Cada punto representa una observación en el conjunto de prueba.
- En el eje Y:
 - 0 = clase “Precio Bajo”
 - 1 = clase “Precio Alto”
- La línea **azul** son los valores reales (y_{test})
- La línea **naranja** son las predicciones del modelo (y_{pred} o $y_{\text{pred_optimo}}$)

¿Qué vemos aquí?

Ambas líneas están **perfectamente alineadas**, una sobre la otra:



No hay ruido visual ni separación entre las dos líneas.



Eso indica que las predicciones **coinciden casi perfectamente con los valores reales**, lo cual puede significar dos cosas:

Interpretación 1: El modelo clasifica muy bien

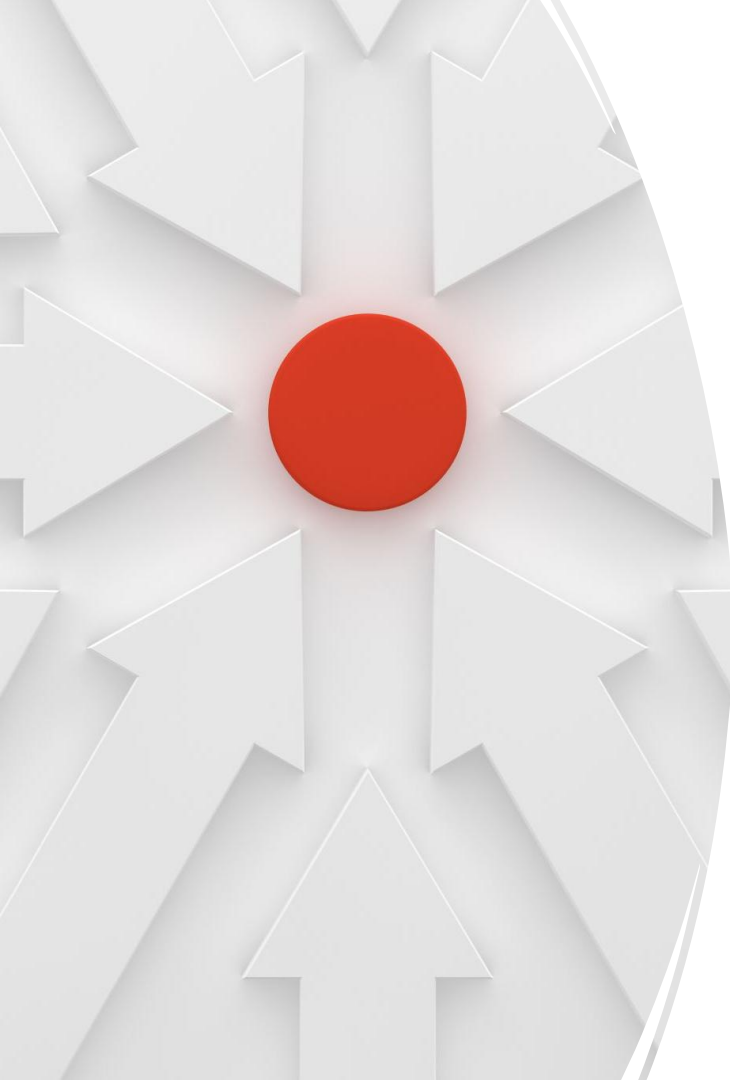
- Si se ha usado el **umbral óptimo (como -0.83)** que maximizaba el F1-score,
- Y si este gráfico muestra un **solapamiento exacto**, entonces el modelo puede estar **clasificando con muy poca diferencia respecto a los valores reales**.
- Es decir, el modelo está siendo **muy preciso y equilibrado**.
- En este caso, eso es **positivo**, aunque el gráfico no sea muy expresivo.

Interpretación 2:

- El gráfico necesita otro enfoque para aportar valor. Este tipo de gráfico funciona mejor cuando hay más separación o ruido en las predicciones. Si todo está bien alineado, puede parecer plano.

Interpretación del modelo SVM

- El modelo es **muy preciso si nos importa evitar falsos positivos** (alta precisión a bajo recall).
- Si nos interesa **detectar la mayoría de los precios altos (alta recall)**, el modelo puede hacerlo, pero **pierde precisión**, lo que significa **más falsos positivos**.



¿Y qué hacer con esto?

- Podemos **elegir un umbral** de decisión según el **punto que mejor se alinee con las necesidades**:
 - ¿Prefieres **no perderte ningún precio alto** (por ejemplo, para no quedarte sin stock)? Elige recall alto, aunque te equivoques más.
 - ¿Prefieres ser **muy seguro de cuándo es alto**, aunque te pierdas algunos? Elige precisión alta.

Elegir alto recall (sensibilidad alta):

- "Quiero **detectar casi todos los precios altos**, incluso si me equivoco y algunos no lo son."
- **Ventaja:** no se escapan los importantes (precios realmente altos).
- **Desventaja:** puedes sobrerreaccionar, hacer ajustes logísticos innecesarios (falsos positivos).
- **Ejemplo:** Prefieres **abastecerte de más frutas** si crees que el precio subirá, aunque algunas no lo hagan.

Elegir alta precisión:

- "Quiero estar **muy seguro de que un producto tiene precio alto** antes de actuar."
- **Ventaja:** tus alertas son confiables (pocos falsos positivos).
- **Desventaja:** se te escapan casos importantes (falsos negativos).
- **Ejemplo:** Solo promocionas un producto como caro si **estás seguro**, aunque algunos productos caros pasen desapercibidos.

¿Cuándo usar SVM?

01

Quando tienes **datos bien separados** o **moderadamente no lineales**.

02

Quando tienes **pocas dimensiones** y **no tantos datos** (SVMs no escalan tan bien con millones de muestras).

03

Quando necesitas buena **precisión** y **generalización**.

Ventajas y desventajas

Ventajas

Funciona bien en espacios de alta dimensión

Eficiente en problemas de clasificación no lineales

Robusto al sobreajuste si se elige bien C y kernel

Desventajas

Lento con grandes volúmenes de datos

Difícil de ajustar el kernel y parámetros

Menos interpretabilidad que regresión lineal



C1 TALLER

- CONTINUANDO CON AREA DE ENTRETANIMIENTO, APLICAR SVM A PROYECTO, TENIENDO EN CUENTA LOS ELEMENTOS DE CLASE.

¡MUCHAS GRACIAS!

